Sukkur Institute of Business Administration University
Department of Computer Science

Web Engineering Theory
BS – V (Software Engineering) – ‘A’
Fall 2026

Instructor: Sir Nisar Ahmed Siddiqui

FlowForge

(A Browser-Based Visual Workflow Automation Platform)

Submission Profile

Final Project Idea Report

Name: Muhammad Anas Qadri **CMS ID:** 053-24-0016

Name: Pirha Mahar **CMS ID:** 053-24-0028

Table of Contents

Executive Summary 3
Problem Statement 3
How FlowForge Addresses This Gap 3
Modules 4
M1 — App Shell (Basic Layout) 4
Deliverable: 4
M2 — Builder Screen (Drag and Connect Boxes) 4
Deliverable: 4
M3 — Box Menu (List of Available Boxes) 4
Deliverable: 4
M4 — Settings Form 5
Key Features: 5
Deliverable: 5
M5 — The Engine (Workflow Execution) 6
Deliverable: 6
M6 — Run History (What Happened Last Time) 6
Deliverable: 6
M7 — Saving Data (Persistence) 6
Deliverable: 6
M8 — Templates & Settings (Finishing Touches) 6
Deliverable: 6

Executive Summary

FlowForge is a browser-based visual workflow automation platform that allows users to build, test, and run automation rules without writing a single line of code. Users drag boxes onto a canvas, connect them with lines to define execution order, and the built-in engine runs the workflow automatically — step by step, again and again.

The entire application runs inside the browser. There is no backend, no server, and no external database. All data workflows, run history, and settings is stored locally using IndexedDB, ensuring full persistence across sessions.

FlowForge is a simplified, self-contained alternative to tools like Zapier and n8n, built entirely with HTML, CSS, and vanilla JavaScript.

Problem Statement

Most automation tools available today (Zapier, n8n, Make) are:

- Cloud-based — require an account, internet connection, and often a paid subscription.
- Complex — steep learning curve for beginners.
- Heavy — rely on frameworks, backend servers, and external databases.
- Closed — users cannot inspect or modify how the engine works.

There is a need for a lightweight, browser-native, zero-dependency automation tool that:

- Runs entirely on the client side.
- Requires no installation, no account, and no backend.
- Is simple enough for beginners but powerful enough for real use cases.

How FlowForge Addresses This Gap

Gap	How FlowForge Solves It
Cloud tools require accounts & internet	FlowForge runs 100% in the browser — no signup, no server.
Existing tools are complex for beginners	FlowForge uses a simple drag-and-drop interface with pre-built templates.
Most tools rely on frameworks & backends	FlowForge is built with vanilla HTML, CSS, and JavaScript only.
Data privacy is a concern	All data stays in the user's browser via IndexedDB — nothing is uploaded.
Automation tools are expensive	FlowForge is free, open-source, and deployable on GitHub Pages.
Learning resources are abstract	FlowForge demonstrates real-world use of drag-and-drop, SVG, IndexedDB, and async execution.

By bringing these features together in one lightweight, browser-based tool, FlowForge provides a practical and easy-to-use platform for creating, managing, and running workflows.

Modules

The project is divided into 8 modules. Each module is built and tested independently, and each build on the previous ones.

M1 — App Shell (Basic Layout)

The overall skeleton of the application.

- Sidebar navigation: Dashboard, Builder, Runs, Templates, Settings
- Responsive layout (desktop + tablet)
- Light and dark mode toggle
- Popups and notification messages

Deliverable:

A working shell with navigation and theme switching.

M2 — Builder Screen (Drag and Connect Boxes)

The core screen where users create workflows. This is the most JavaScript-heavy part of the project.

- Drag and drop boxes anywhere on the canvas
- Draw SVG lines between boxes to connect them
- Select, move, delete, and copy boxes
- Undo/redo functionality added.
- Zoom and pan around the canvas

Deliverable:

A fully interactive canvas with draggable boxes and connectable lines.

M3 — Box Menu (List of Available Boxes)

The Box Menu is a side panel that displays all available boxes. The boxes are grouped into four categories based on their purpose.

Category	Box	Purpose
Start	Manual Start	Starts the workflow manually.
	Timer (Schedule)	Starts the workflow at a scheduled time.
Action	HTTP Request	Sends a request to a specified URL.
	Transform	Processes or changes data.
	Set Variable	Creates or updates a variable.
	Wait	Pauses the workflow for a specified time.
Decision	Condition (Yes/No Branch)	Checks a condition and selects a path.
	Loop (Repeat)	Repeats a part of the workflow.
Output	Send Notification	Shows a notification inside FlowForge.
	Write to Log	Records information in the workflow log.

Deliverable:

A categorized, searchable box menu.

M4 — Settings Form

When a user clicks on a box, a settings form opens on the side panel. This form allows the user to configure the box's behaviour.

Key Features:

1. Dynamic Form Fields:

The form changes automatically based on the selected box type.

Example:

- An HTTP Request box shows fields for URL, method, headers, and body.
- A Set Variable box shows fields for variable name and value.
- A Condition box shows fields for comparison operators and values.

2. Input Validation:

Each field is validated before the workflow can be saved.

Example:

- URL fields must start with http:// or https://.
- Required fields cannot be left empty.
- Numeric fields must contain valid numbers.

3. Variable Interpolation ({{variable}})

FlowForge allows users to use values from one box in another box by using double curly braces, such as {{website}} or {{name}}. During workflow execution, FlowForge replaces the variable with its actual value.

Example:

Box	Type	Field	Value
Box 1	Set Variable	Variable Name	website
Box 1	Set Variable	Value	https://example.com
Box 2	HTTP Request	URL	{{website}}

At runtime:

- Box 1 sets the value of website to https://example.com.
- Box 2 replaces {{website}} with the stored value.
- The HTTP Request is sent to https://example.com.
- This feature makes workflows more dynamic and reusable, as values can be changed without modifying the workflow structure.

Deliverable:

A working form system with:

- Dynamic fields based on box type
- Input validation
- {{variable}} interpolation from earlier boxes

M5 — The Engine (Workflow Execution)

The Engine is the main part of FlowForge that reads the boxes and their connections and executes the workflow in the correct order.

- Topological sorting to determine the execution order
- Cycle detection to prevent infinite loops
- Separate handlers for different box types
- Data passing between boxes
- Error handling, retries, and timeouts

Deliverable:

A working workflow engine that can execute valid workflows.

M6 — Run History (What Happened Last Time)

Run History keeps a record of previous workflow executions and their results.

- List of previous runs with status and duration
- Step-by-step timeline for each run
- Input and output data for each step
- Error messages and stack traces

Deliverable:

A Run History page showing detailed execution logs.

M7 — Saving Data (Persistence)

FlowForge stores application data in the browser using IndexedDB. This allows users to save their work and access it again when they return to the application.

- Save and load workflows, run history, and settings
- Export workflows as JSON files
- Import workflows from JSON files
- Automatically save changes

Deliverable:

Full persistence with import/export.

M8 — Templates & Settings (Finishing Touches)

Extra features for usability.

- Pre-built workflow templates
- Settings page (theme, defaults)
- Clear all data option

Deliverable:

Templates and settings page.